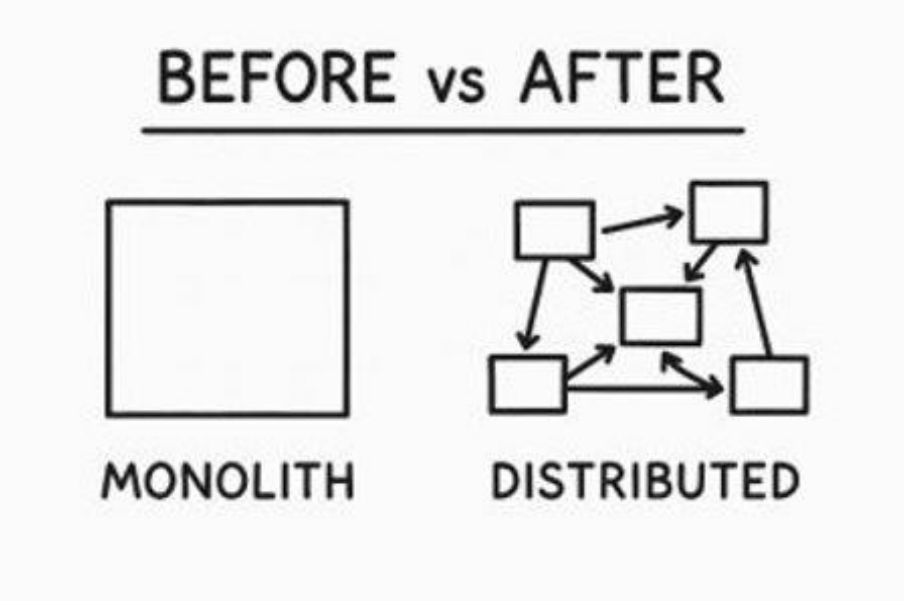
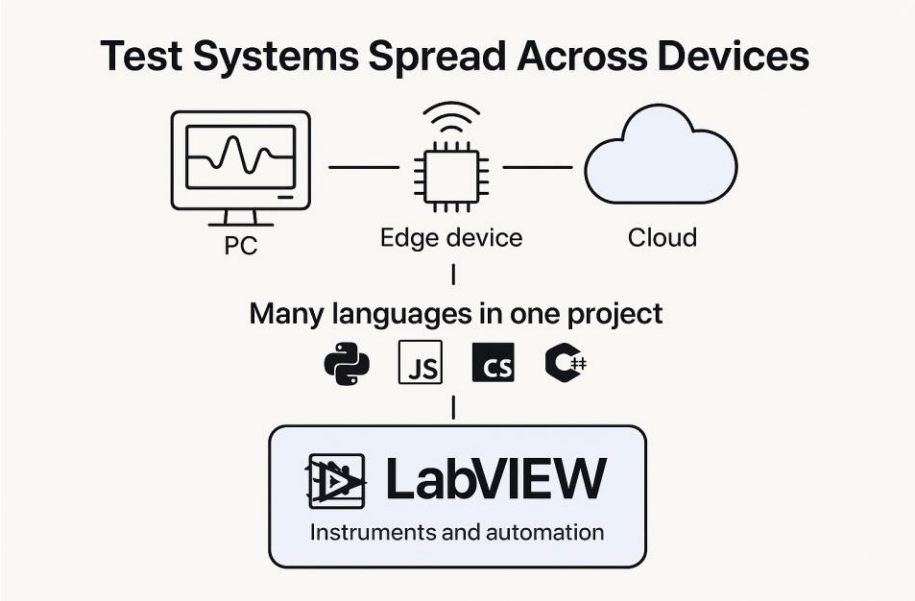
LabVIEW in a Polyglot World

Building Scalable Multi-Language Frameworks Using gRPC

The Big Picture (Why This Matters)

- Test systems are no longer single apps. They spread across PCs, edge devices, and the cloud.
- Teams now use many languages in one project.
- LabVIEW is great for instruments and automation—but it must work smoothly with other tools.





Soliton
Celebrating 25 years

www.solitontech.com

Who Am I?

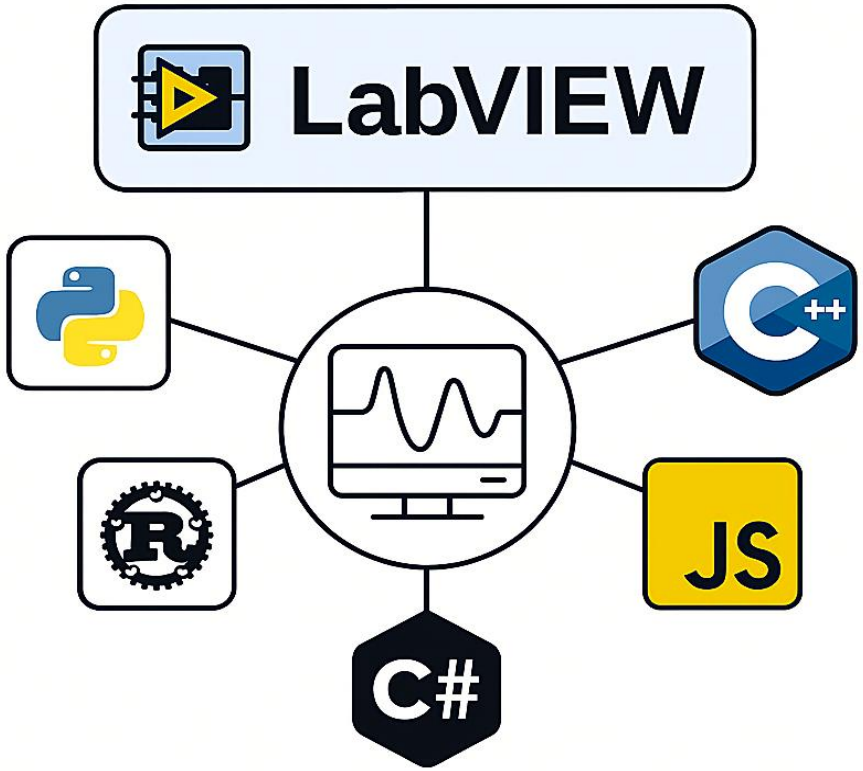
Kingsly Andrews M

Senior Project Engineer, Soliton Technologies Pvt Ltd
Certified LabVIEW Developer

3yr+: LabVIEW, NI DPE
2yr+: LV gRPC, NI Scope
1yr+: LabVIEW FPGA
Protocols: SWI3S, SW, I2S, I3C, I2C, CAN(classic, FD, XL)

What “Polyglot” Means (In Simple Terms)

- Polyglot = many languages working together in one system.
- Use each language where it shines:
  - Python for AI/ML and data analysis
  - C++ for fast algorithms
  - JavaScript for dashboards
  - Rust/Go/C# for services and tools
- The trick is to connect them well.



Meet gRPC (The Connector)

- In 2015, Google open-sourced their project which eventually would be the one called gRPC.
- gRPC is a cross-platform open-source high performance remote procedure call framework.
- RPCs allow you to write code as though it will be run on a local computer, even though it may be executed on another computer.
- A RPC is a form of Client-Server Communication that uses a function call rather than a usual HTTP call.

What makes gRPC so popular?

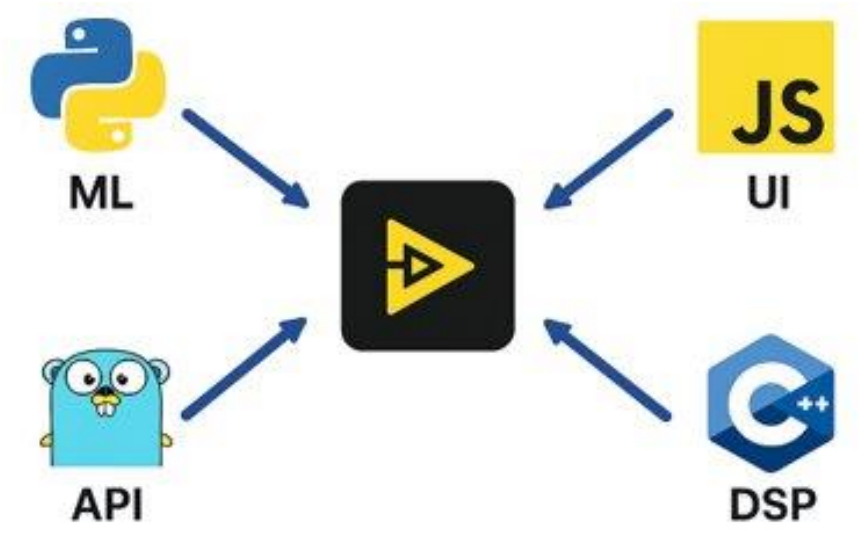
- A fast way for programs to call functions across machines.
- Abstraction is easy (it's a function call)
- Works with many languages out of the box.
- Uses .proto files (like a contract) so both sides agree on names, types, and messages.
- HTTP calls are often confusing, so this makes it easier

Why gRPC + LabVIEW Is a Good Match

- **Clear boundaries:** LabVIEW does orchestration; plugins do their specialty.
- **Future-proof:** You can swap a Python plugin for C++ without changing the LabVIEW diagram.
- **Consistent:** Contracts live in version control; teams align on the same interface.
- **Flexible:** Runs on the same PC, a server, or in containers—no diagram changes.

A Simple Reference Architecture

- **Core (LabVIEW):** sequences, hardware control, test logic.
- **Services/Plugins:** Python (ML), C++ (algorithms), Go (APIs), JS (UI).
- **Contract:** Shared .proto files define requests and responses.
- **Deployment:** Local, remote, or containers—same interface.



What are Protocol Buffers?

- Protocol buffers provide a language-neutral, platform-neutral, extensible mechanism for serializing structured data in a forward-compatible and backward-compatible way.
- It's like JSON, except it's smaller and faster, and it generates native language bindings.

```
syntax = "proto3"; // Specifies that this file uses Protobuf version 3 syntax
package Communication;
// Defines the Protobuf namespace for all messages/services in this file
message Person {
  optional string name = 1;
  optional int32 id = 2;
  optional string email = 3;
}
```

What Goes Into the .proto (Plain English)

- **Service names:** what area/function this covers (e.g., HelloService, MeasurementService).
- **Messages:** input and output data with types (e.g., HelloRequest, HelloResponse, MeasureData).
- **Methods:** actions like SayHello, StartTest, GetStatus, FetchResults.

```
service HelloService {
  rpc SayHello (HelloRequest) returns (HelloResponse);
}

message HelloRequest {
  string greeting = 1;
}

message HelloResponse {
  string reply = 1;
}
```

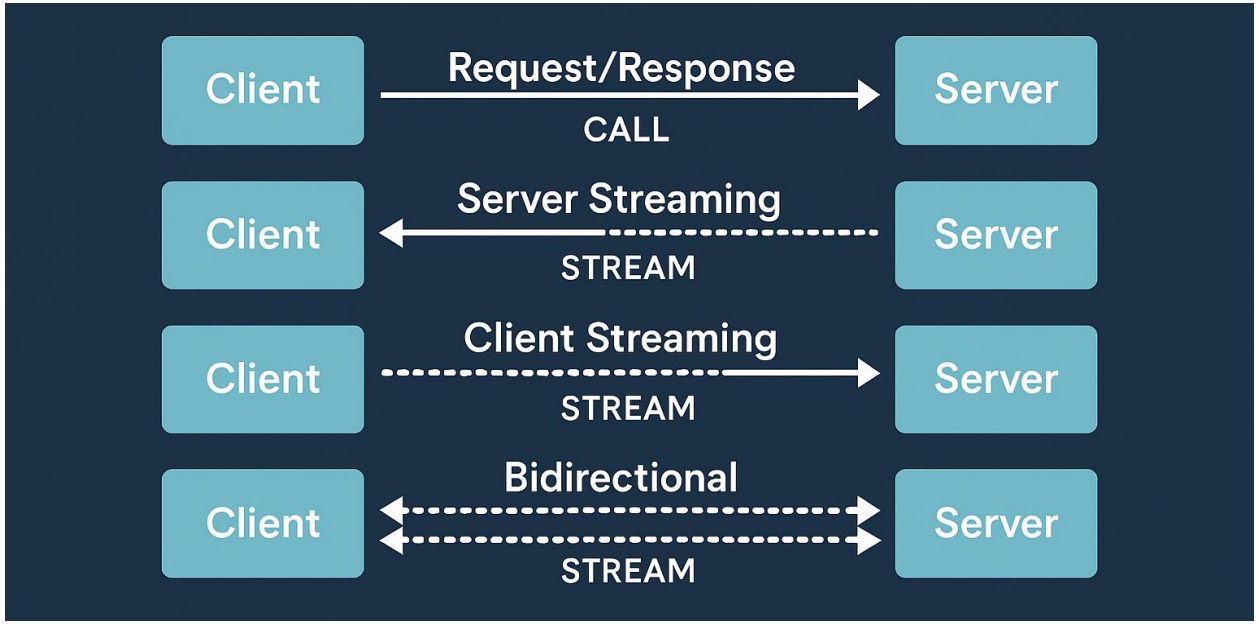
Proto File Datatypes

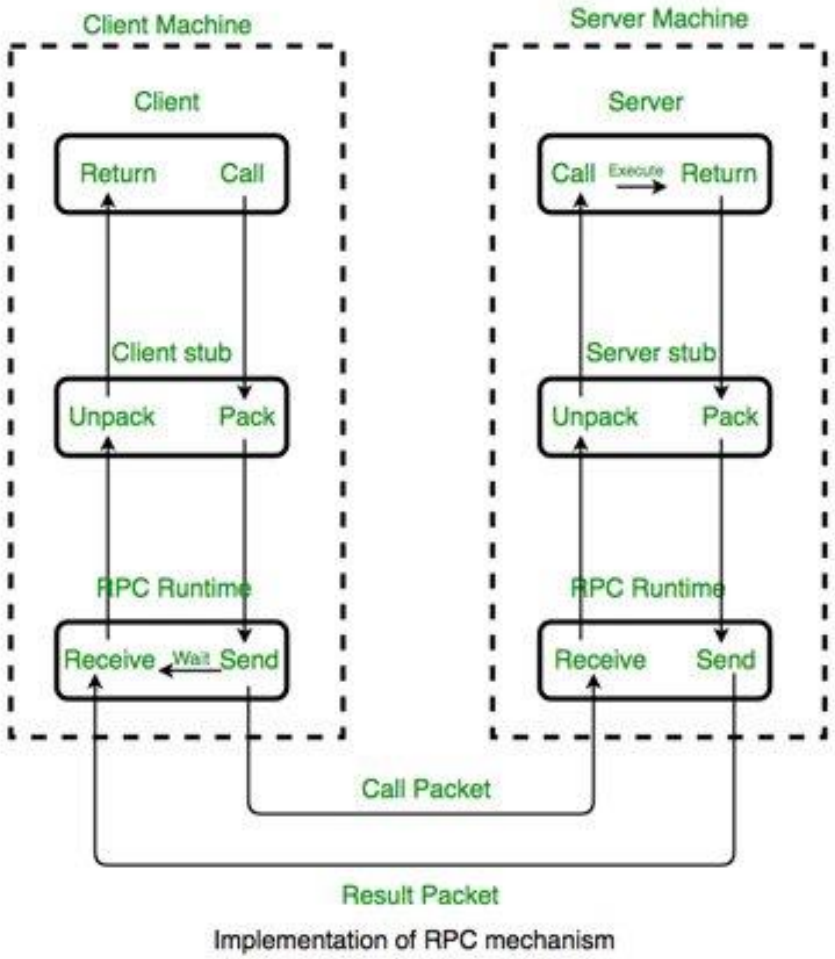
.proto Type	C++ Type	Java/Kotlin Type[1]	Python Type[3]	Go Type	C# Type	PHP Type	Dart Type
double	double	double	float	float64	double	float	double
int32	int32	int	int	int32	int	integer	int
int64	int64	long	int/long[4]	int64	long	integer/string[6]	Int64
uint32	uint32	int[2]	int/long[4]	uint32	uint	integer	int
uint64	uint64	long[2]	int/long[4]	uint64	ulong	integer/string[6]	Int64
bool	bool	boolean	bool	bool	bool	boolean	bool
string	string	String	str/unicode[5]	string	string	string	String

- For strings, the default value is the empty string.
- For bools, the default value is false.
- For numeric types, the default value is zero.
- For enums, the default value is the **first defined enum value**, which must be 0.
- For Arrays, add "repeated" key in front of the datatype.

Communication Patterns That Work

- **Request/Response:** call a function and get a result.
- **Server Streaming:** service pushes progress updates.
- **Client Streaming:** send many items before getting a result.
- **Bidirectional:** both sides stream (advanced, use only if needed).

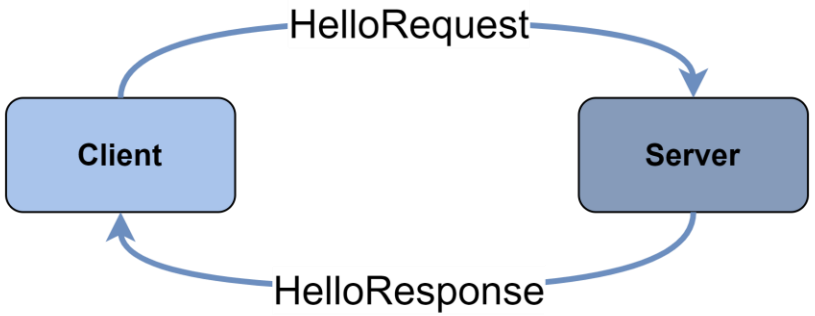




Remote Procedure Call

Unary RPC

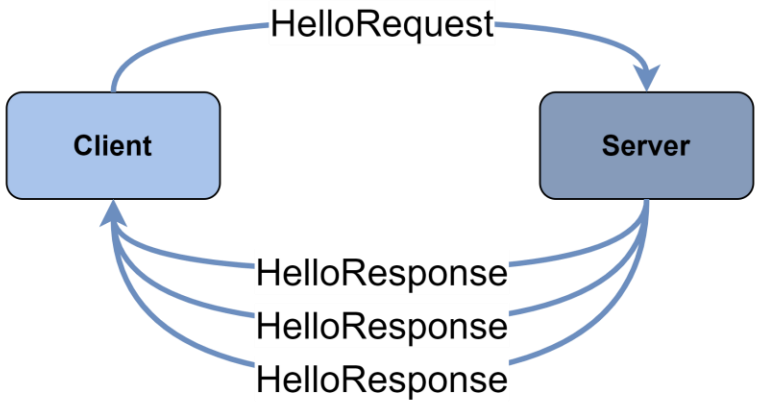
- RPC can be very simple with one request from the client and one response from the server.
- The client starts the RPC by calling a stub method.
- The server receives:
  - The method name
  - The client's metadata
  - The deadline, if one is set
- This type of RPC is called a Unary RPC.
- Unary RPCs work like a normal function call with one input and one output.




```
rpc SayHello(HelloRequest) returns (HelloResponse);
```

Server streaming RPC

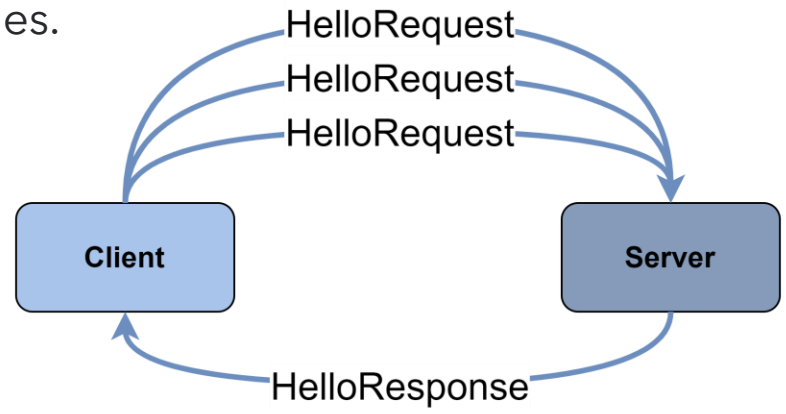
- In a server-streaming RPC, the client sends one request to the server.
- The server responds with a stream of multiple messages instead of a single response.
- After sending all messages, the server sends:
  - A status code
  - An optional status message
  - Optional trailing metadata
- This marks the end of processing on the server side.
- The client reads messages one by one from the stream until no more messages are available.
- gRPC ensures that messages are received in the correct order within the same RPC call.




```
rpc LotsOfReplies(HelloRequest) returns (stream HelloResponse);
```

Client streaming RPC

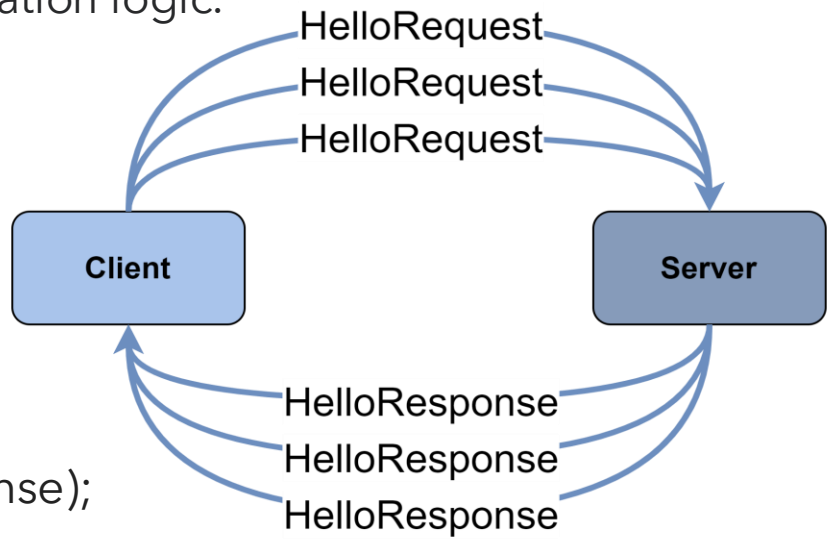
- In a client-streaming RPC, the client sends multiple messages to the server.
- Instead of a single request, the client streams a sequence of messages.
- The server sends back one final response.
- The server response includes:
  - A status code
  - An optional status message
  - Optional trailing metadata
- The server usually replies after receiving all client messages, but it is not mandatory.
- The client waits for the server's response after finishing sending messages.
- gRPC guarantees that all messages are received in order within the same RPC call.




```
rpc LotsOfGreetings(stream HelloRequest) returns (HelloResponse);
```

Bidirectional streaming RPC

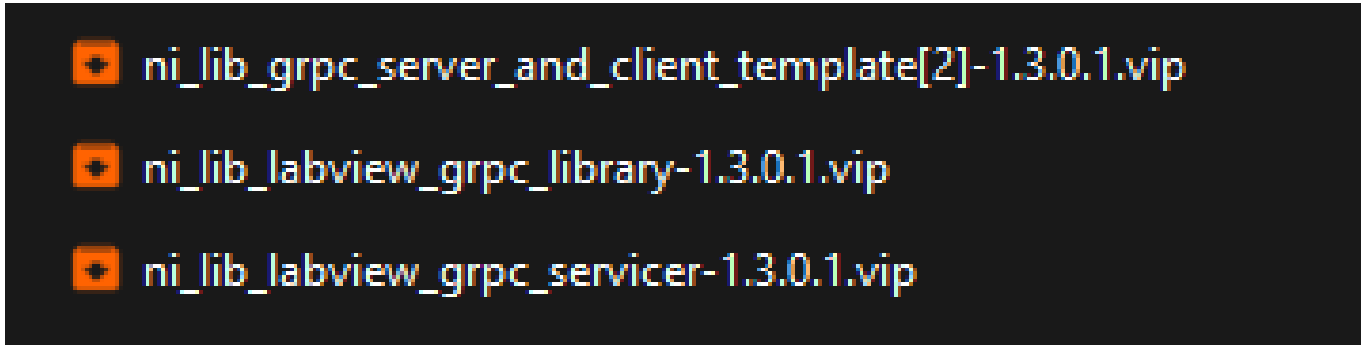
- In a bidirectional streaming RPC, the client starts the call by invoking the method.
- The server receives the client metadata, method name, and deadline.
- The server can send initial metadata immediately or wait until the client starts sending messages.
- Both the client and server can send multiple messages using a read–write stream independently of each other.
- Messages can be sent and received in any order, based on application logic.
- Example interactions:
  - Server waits for all client messages, then responds.
  - Exchange messages back and forth like "ping-pong".
- This type of RPC allows real-time, two-way communication.

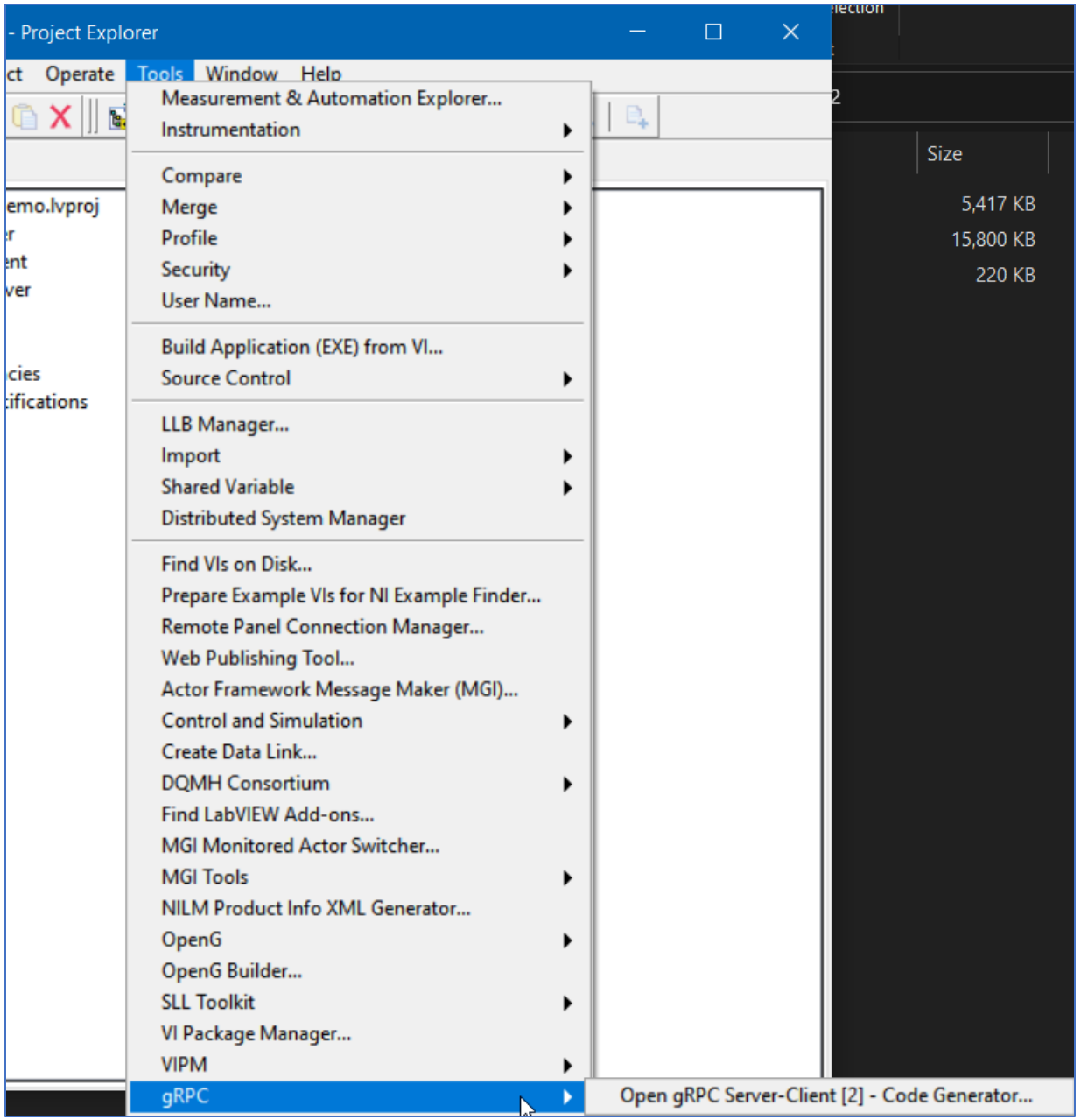


rpc BidiHello(stream HelloRequest) **returns** (stream HelloResponse);

GRPC Setup in labVIEW

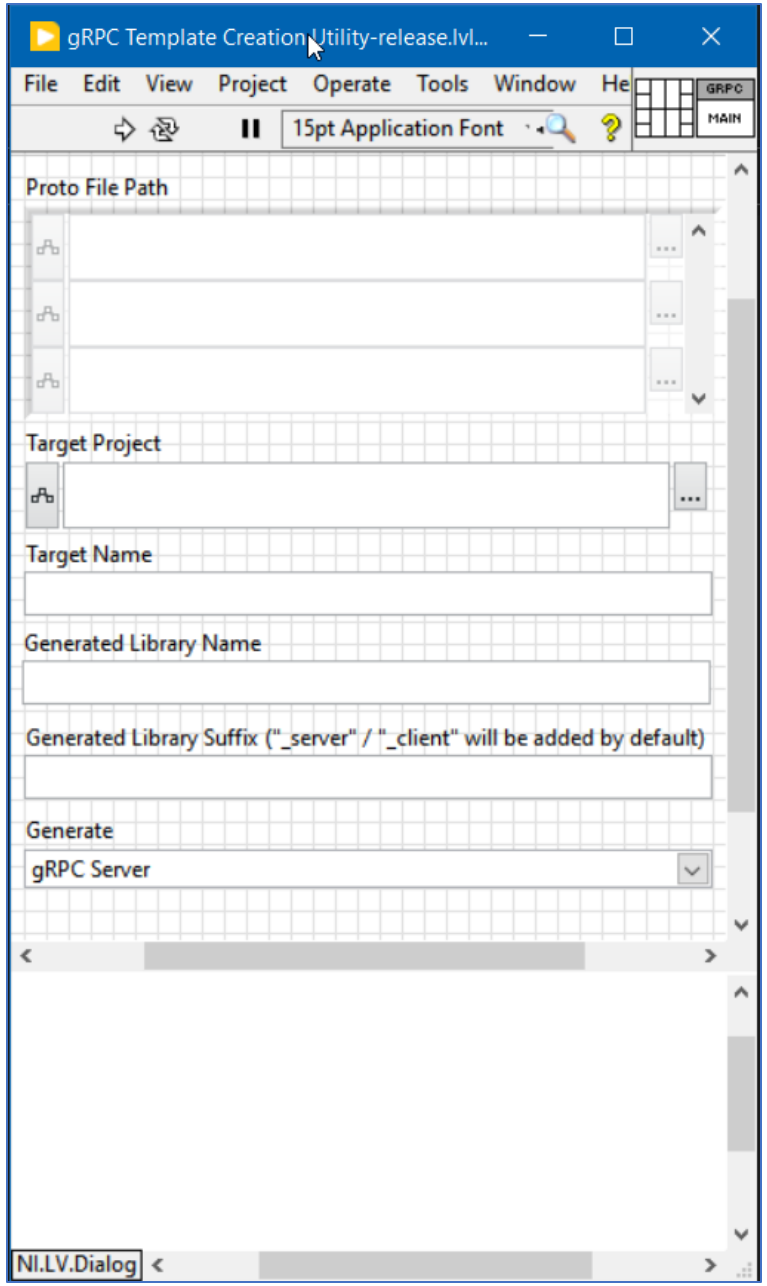
- GitHub - ni/grpc-labview: gRPC client and server support for LabVIEW · GitHub
  This repo contains necessary C++ code and support VIs to implement a gRPC server in LabVIEW.
- Minimum Compatible LabVIEW Version
  - LabVIEW source is saved with LabVIEW 2019.
- Install the Required version of LabVIEW gRPC.
- Here we have used gRPC 1.3.0.1. Double click or open the package using VIPM and click add to library and install.
- The order of installation is
  - Library
  - Servicer
  - Template





gRPC Template Creator

- After installation, gRPC nodes appear in the LabVIEW Functions tab.
- These nodes can be used to build gRPC-based communication in LabVIEW.
- LabVIEW also provides an auto-generation method for gRPC APIs.
- This method uses LabVIEW scripting to generate gRPC APIs based on specific requirements.
- Both approaches help simplify client–server communication setup in LabVIEW.

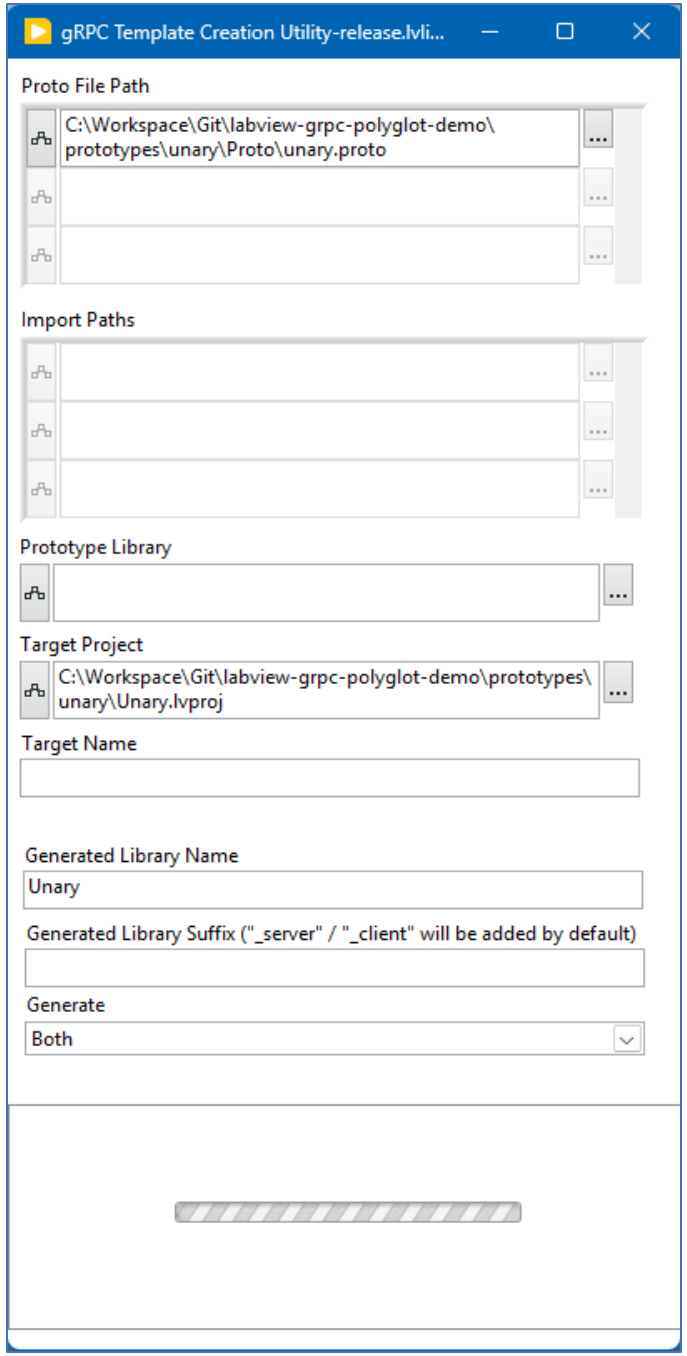


gRPC Template Creator

- The code generator helps create custom client and server APIs in LabVIEW.
- To auto-generate LabVIEW code, two main things are required:
  - A ProtoBuf (.proto) file
  - A LabVIEW project to hold the generated code
- The ProtoBuf file defines the communication structure used by gRPC.
- This file is used to generate:
  - gRPC APIs
  - Controls
  - Classes
  - User events

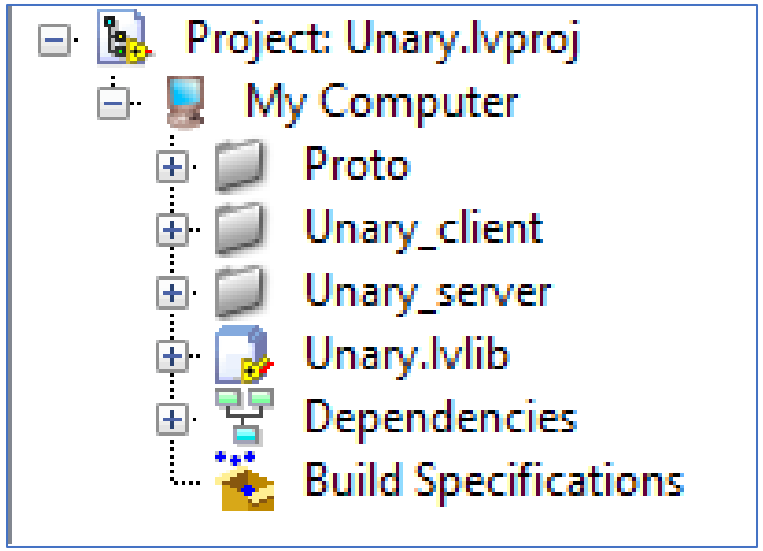
Proto File for Unary RPC Call

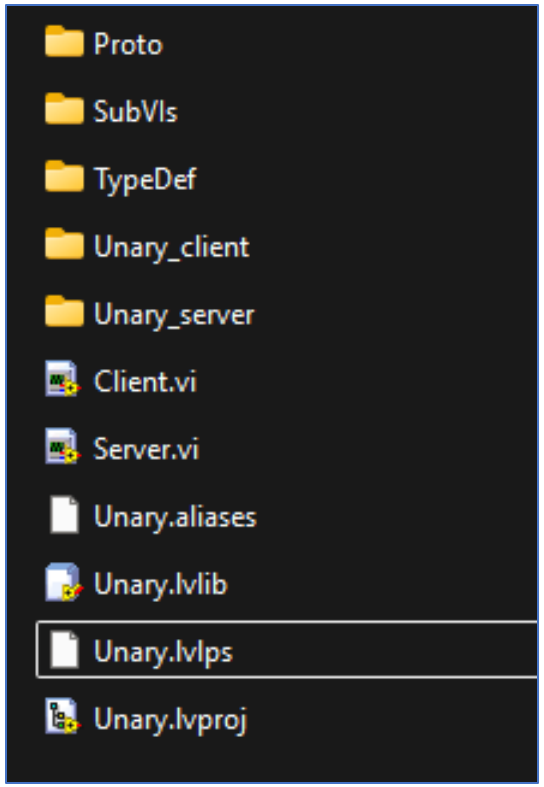
```
syntax = "proto3";

package Protocol;

service I2C
{
  rpc Configure(Request) returns (Response) {}
}
message Request
{
string Request=1;
string Data=2;
}
message Response
{
Request Operation=1;
bool Status=2;
}
```



Generating API's

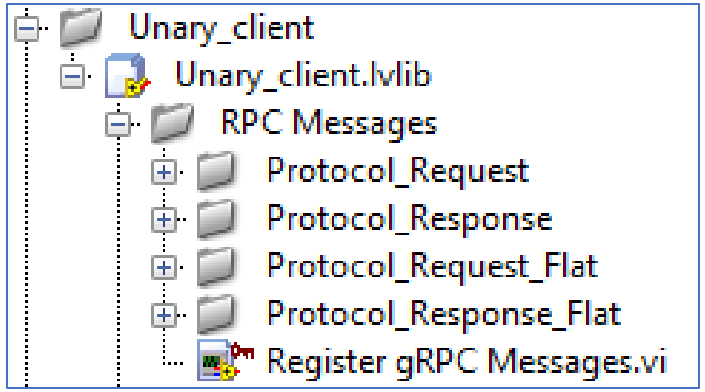
- The files created will be as below parallel to the project file.



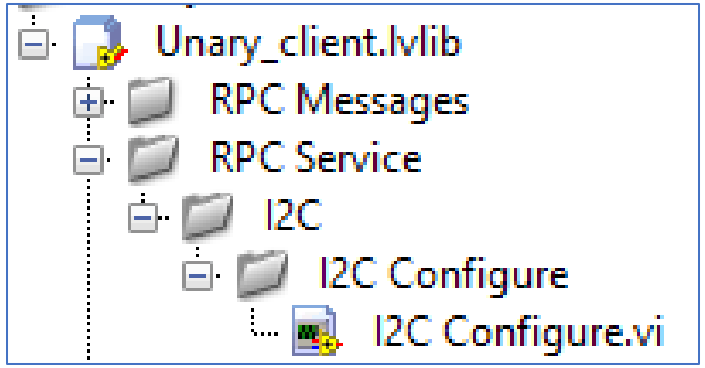


LabVIEW – Client

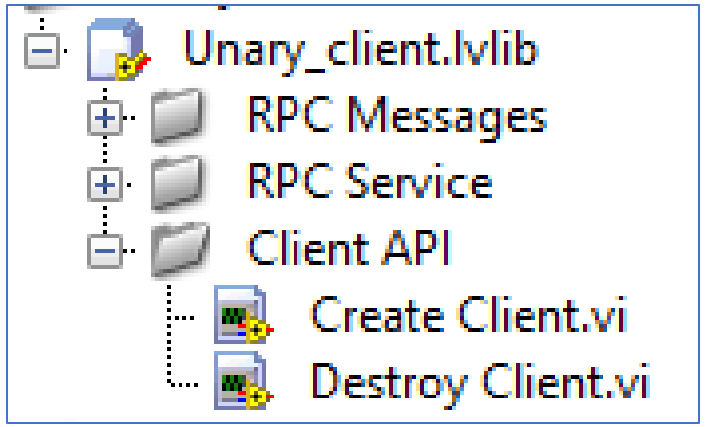
- Client Files. Message will have all the controls.



- Services will have all the services and their API method which can be used to request and receive the response.

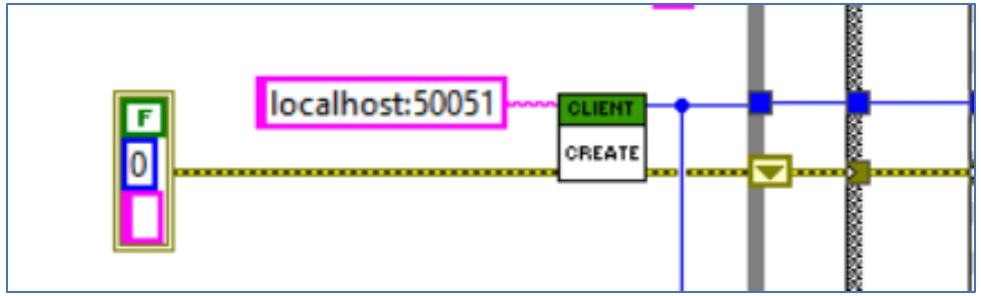


- These APIs can be used for client setup using LabVIEW.

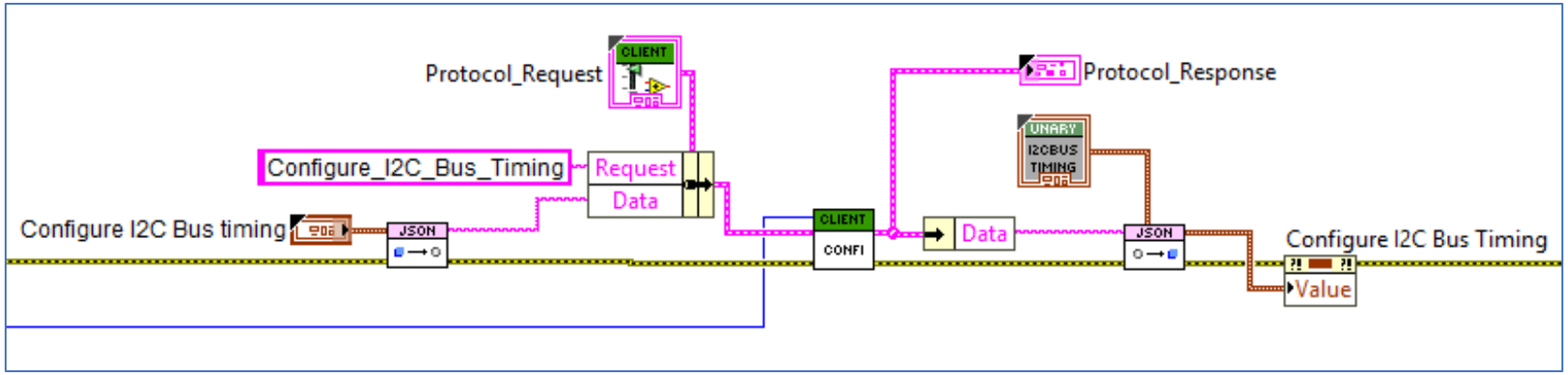


LabVIEW – Client Code

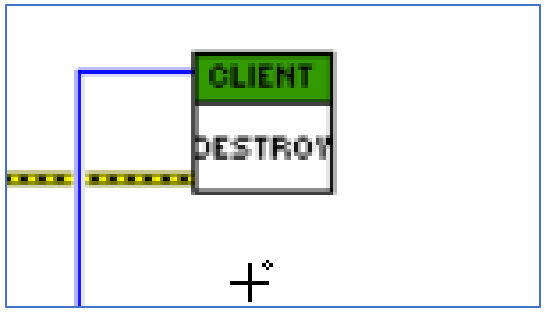
- To create Client.



- To communicate from the client side.

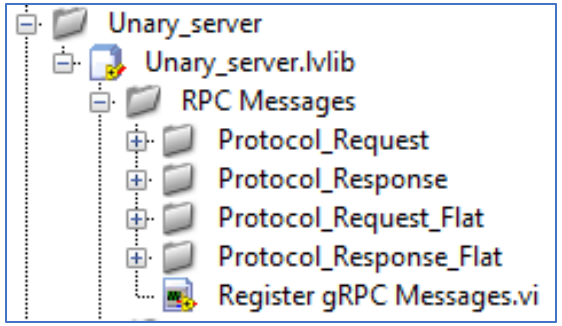


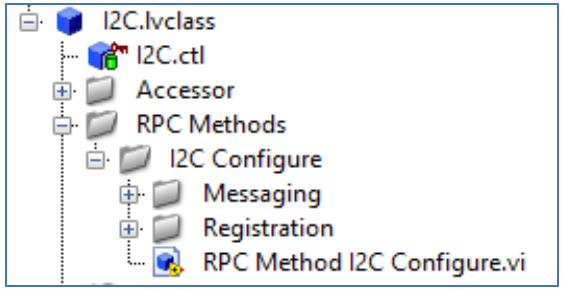
- To Destroy Client.

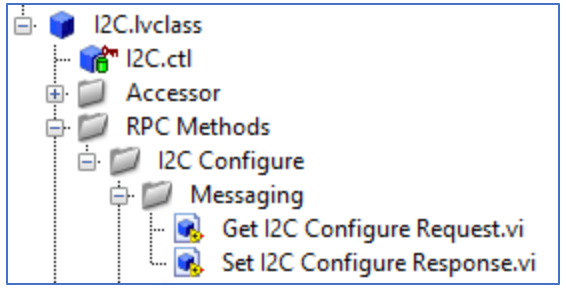


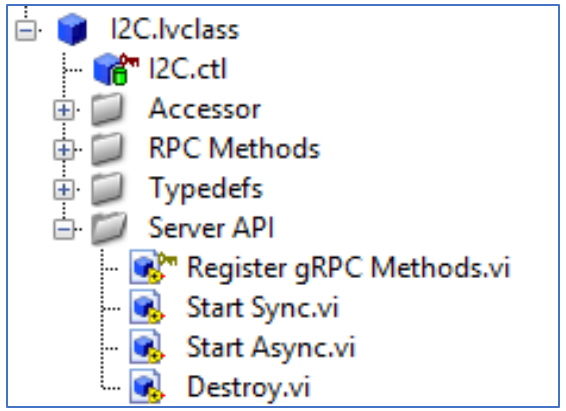
LabVIEW – Server

- Server Files. Message will have all the controls.
- Accessors will contain files from accessing class data. Methods will have registration of messages and the sample VI which can be modified to capture and set the required response. And other typeDefs
- We can use the Get and Set VIs to receive and send message from server side.
- These APIs can be used to setup the server in LabVIEW.



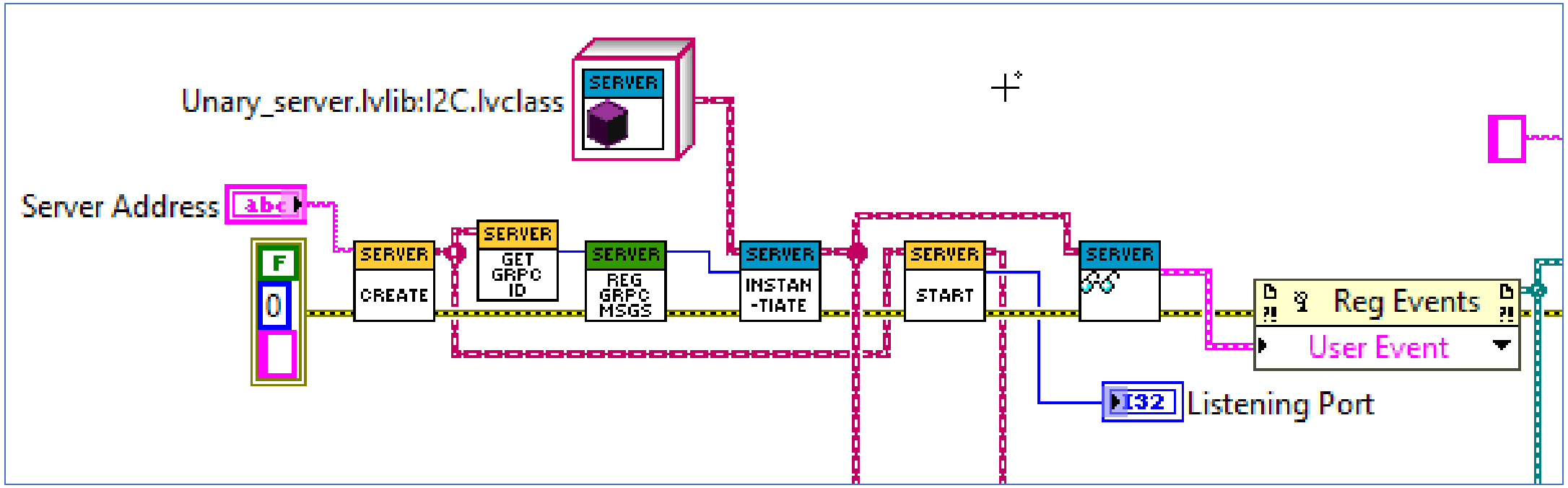






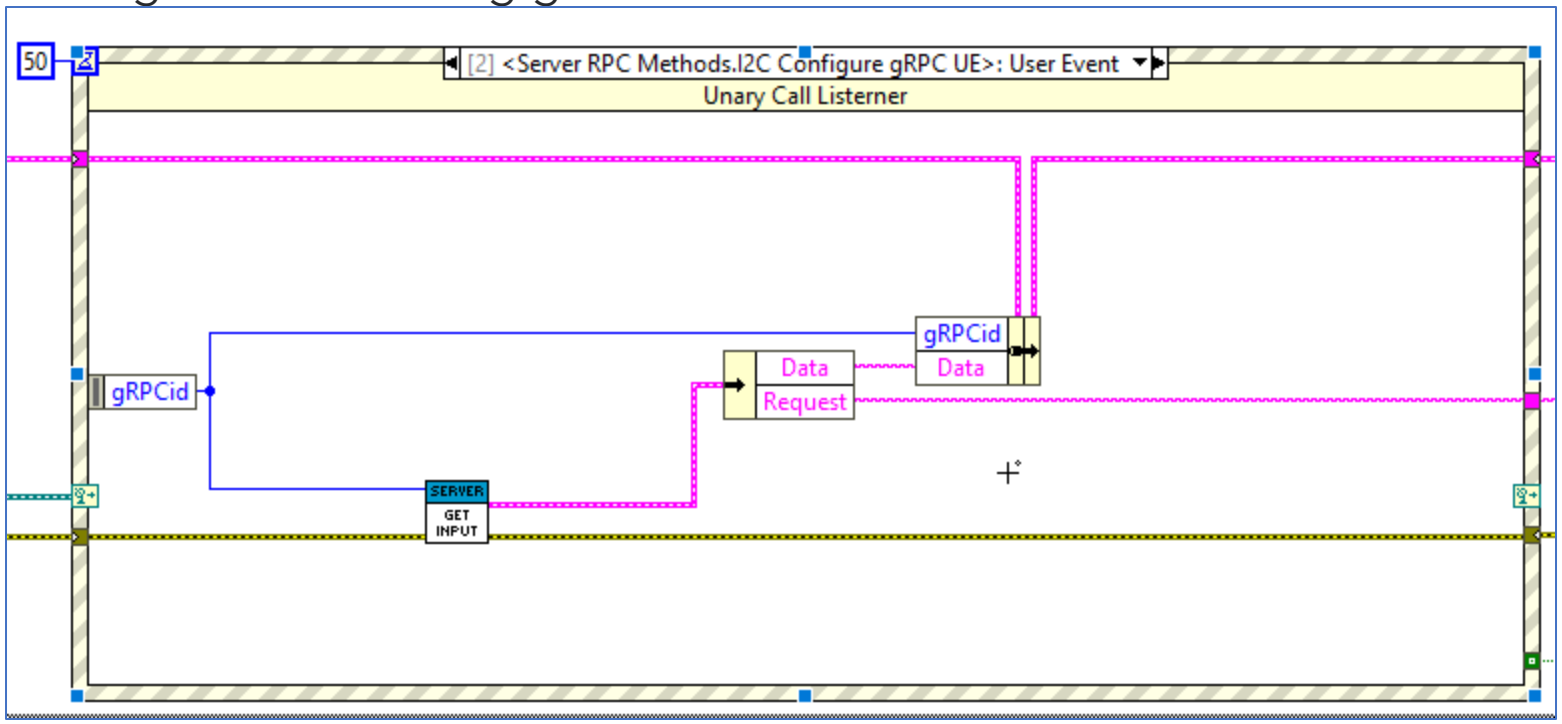
LabVIEW – Server Code

- To setup a custom server with state machine we can follow as below.



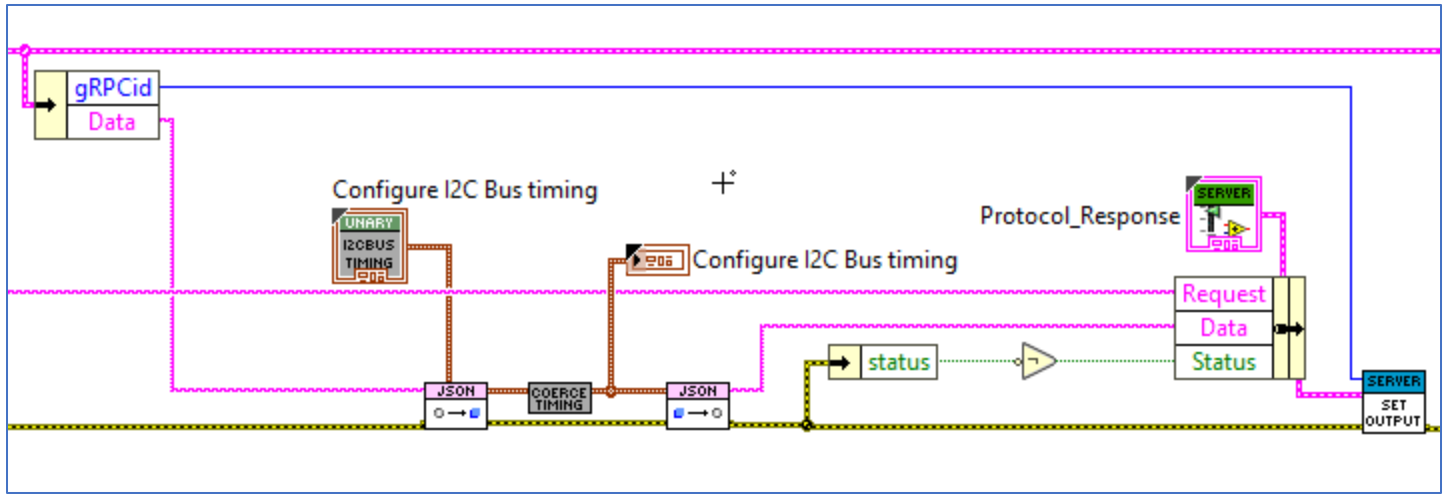
LabVIEW – Server Code

- First create the server. Get the ID and register the messages. Here the class object constant is used as there is only one service named "I2C". Instantiate the service. Start the server. Read the User events and register.
- To get the message in server using generated user event.

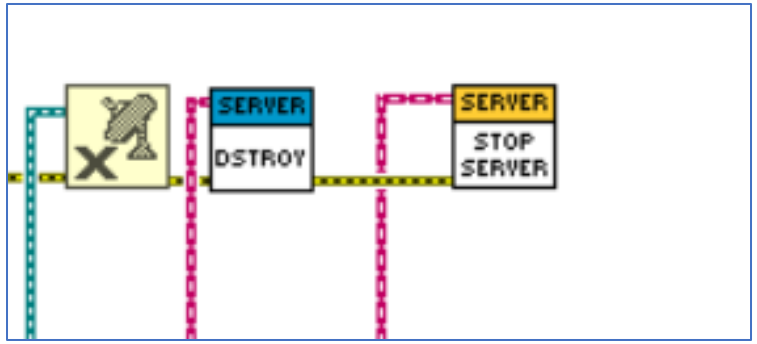


LabVIEW – Server Code

- To set the message in server.



- To stop the server.



- Example Reference.[Kingsly1921/labview-grpc-polyglot-demo: LabVIEW gRPC examples and a polyglot demo for building scalable multi-language systems.]

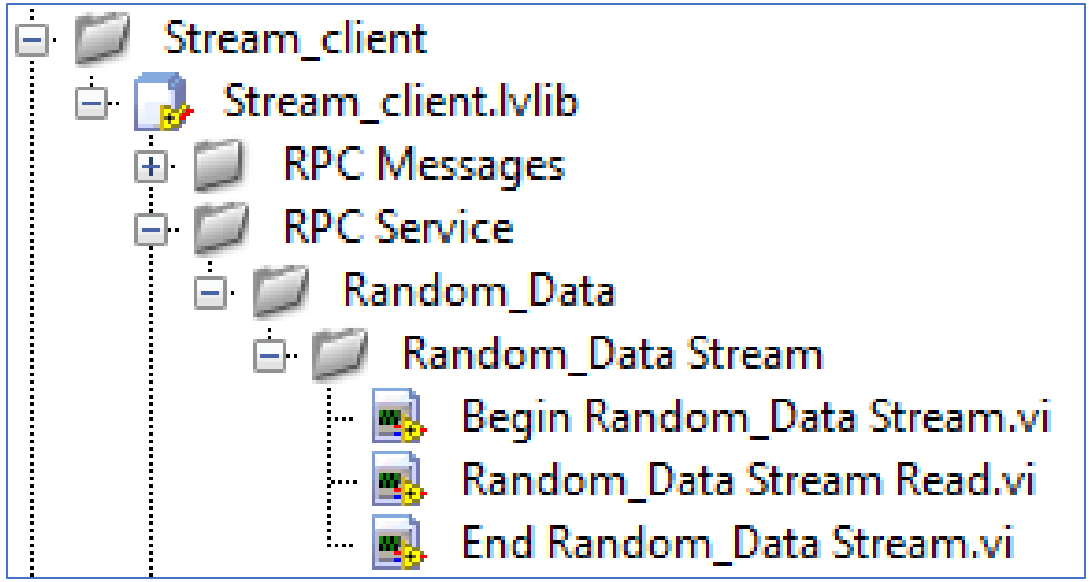
Proto File for Server Stream RPC Call

The proto file response should be added a syntax “stream” at return variable.

```
syntax = "proto3";

package Acquisition;

service Random_Data
{
rpc Stream(Control_Stream) returns (stream Stream_Data){}
}
message Control_Stream
{
int32 Count=1;
}
message Stream_Data
{
int32 Data=1;
bool Status=2;
}
```

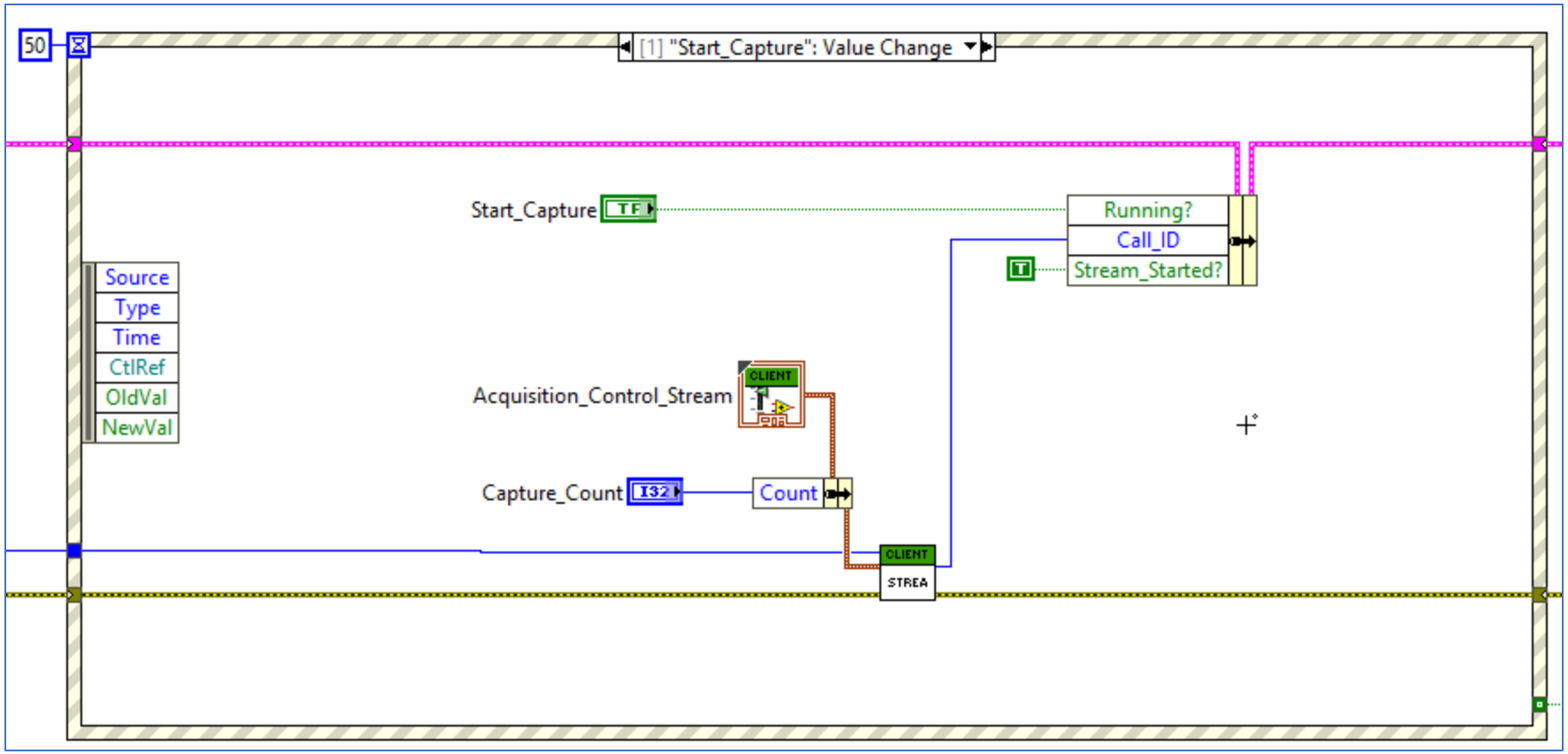
LabVIEW – Client

- In the client stream consist of begin, read and end.



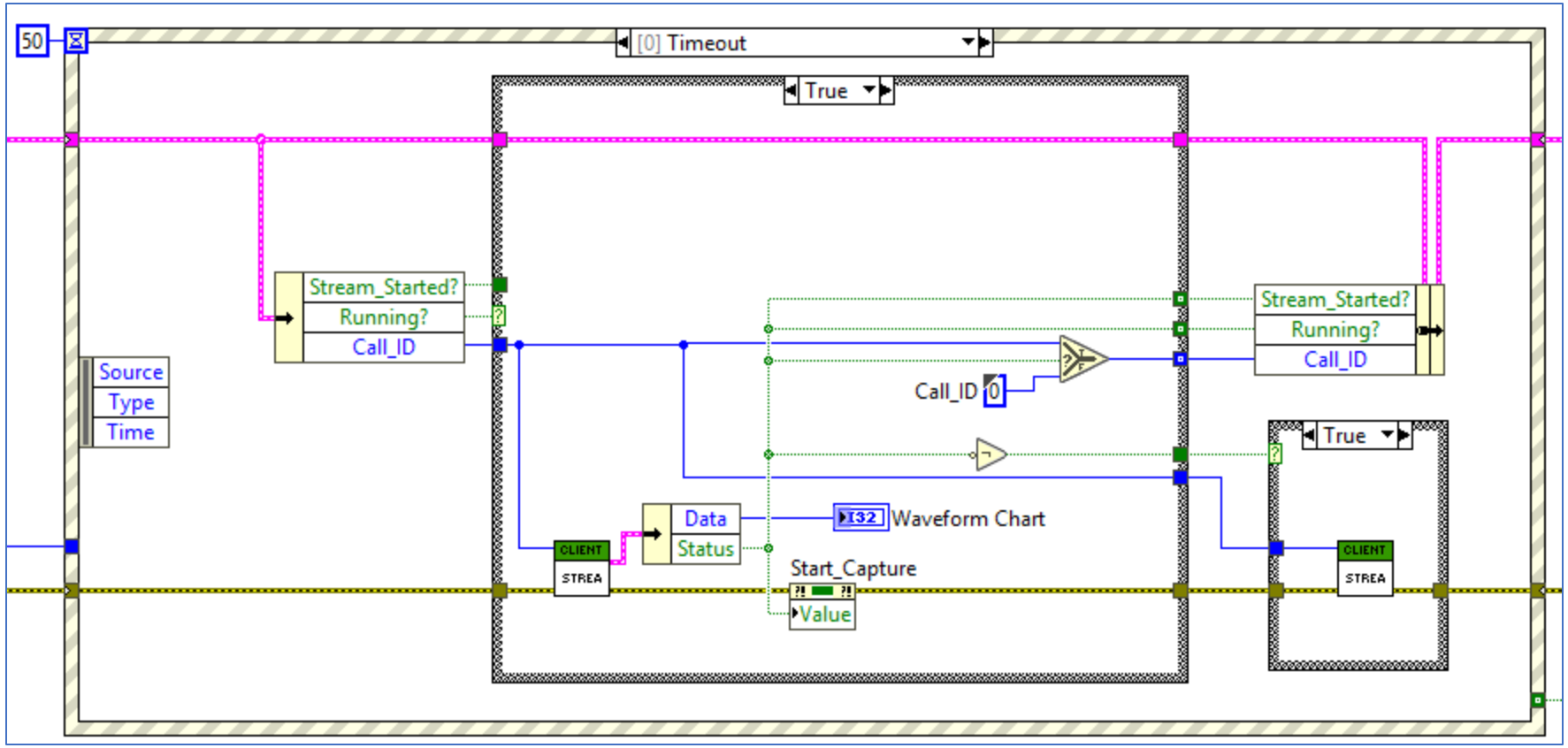
LabVIEW – Client Code

- To begin the stream.



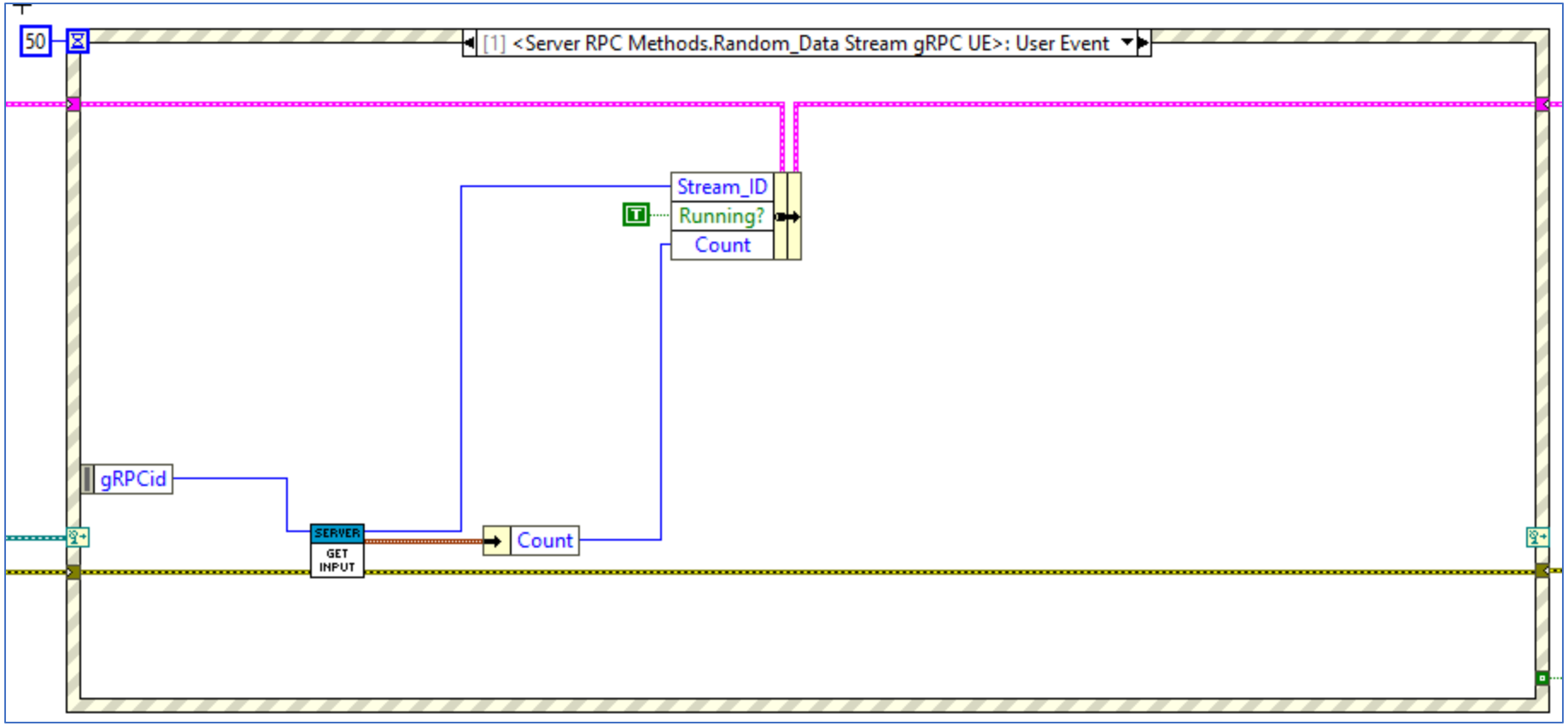
LabVIEW – Client Code

- To read and consistently monitor and end the stream from client and server side.



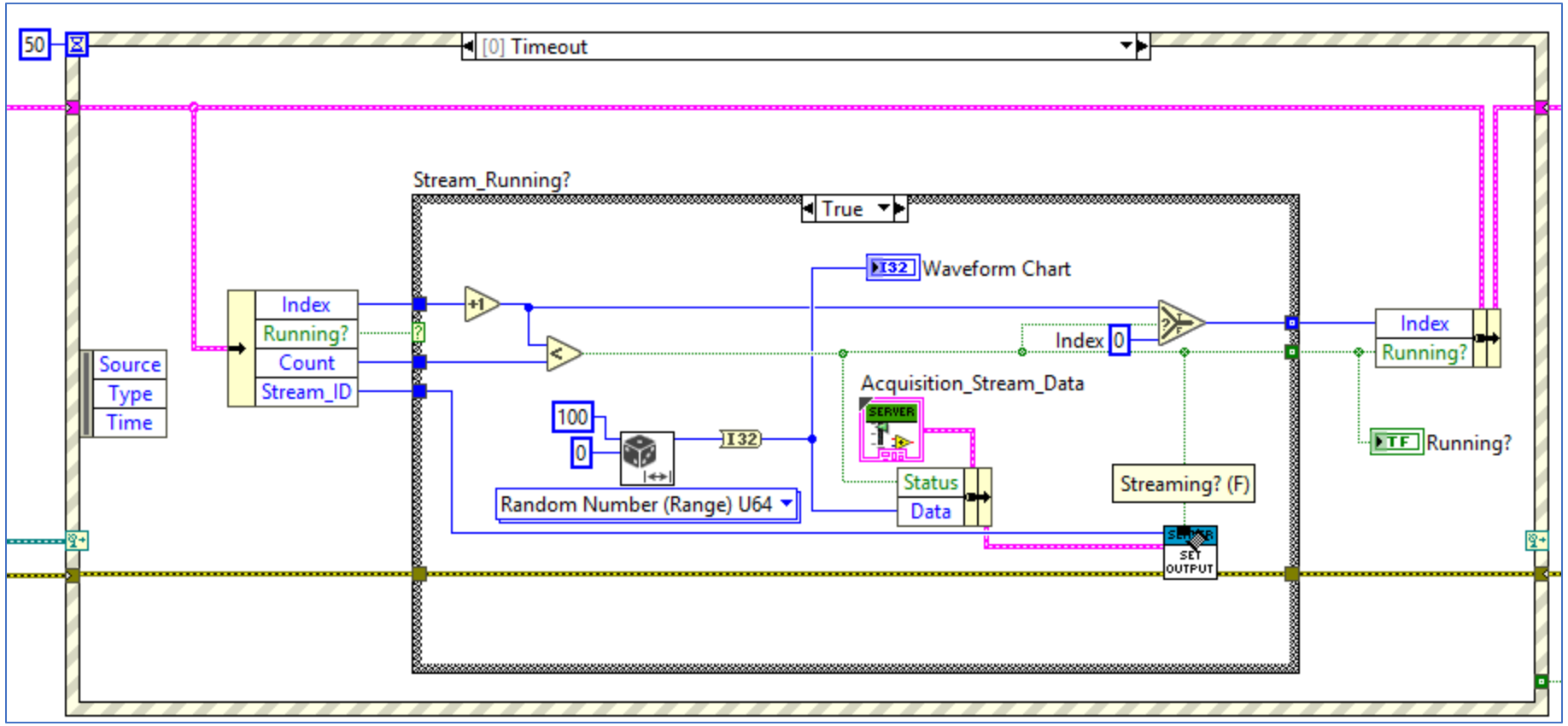
LabVIEW – Server Code

- At the server side the APIs are all similar.
- When the stream begins the event will be triggered.



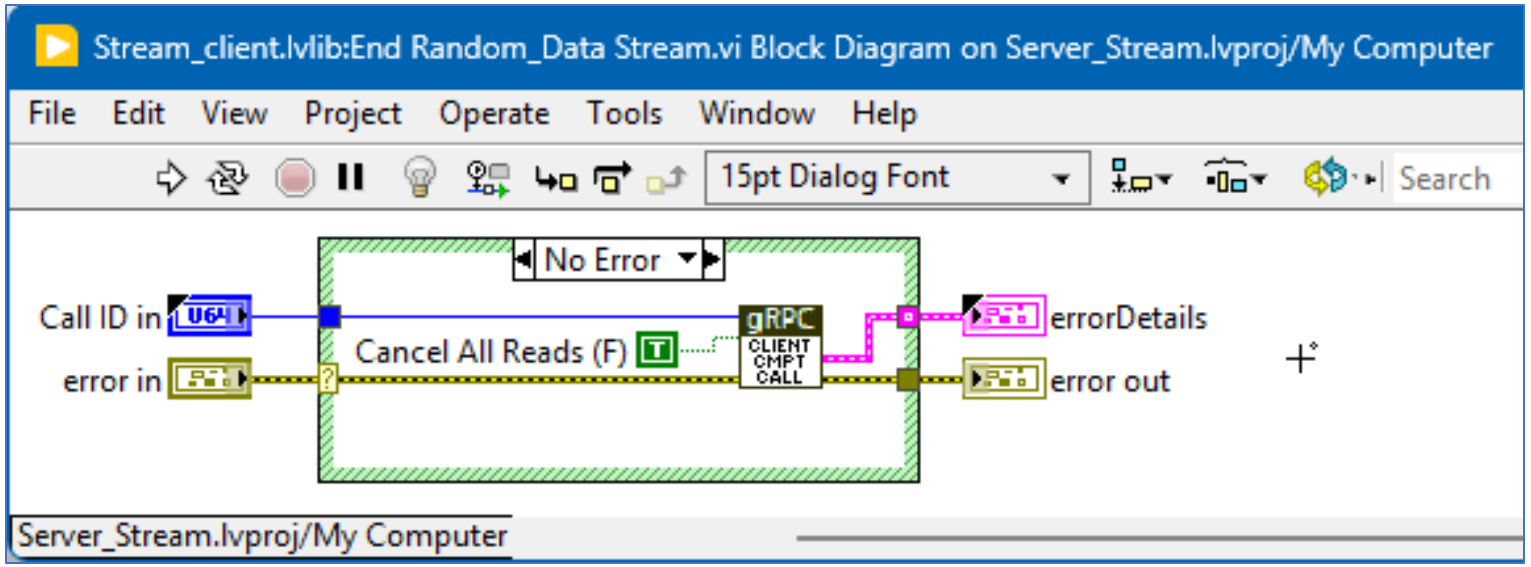
LabVIEW – Server Code

- To stream the value set the true constant is given.



LabVIEW – Client Code

- NOTE: when using stream, the on-client side the end stream will wait for all the values to be read before ending we need to modify it inside the API.
- Refer this example. [Kingsly1921/labview-grpc-polyglot-demo: LabVIEW gRPC examples and a polyglot demo for building scalable multi-language systems.]

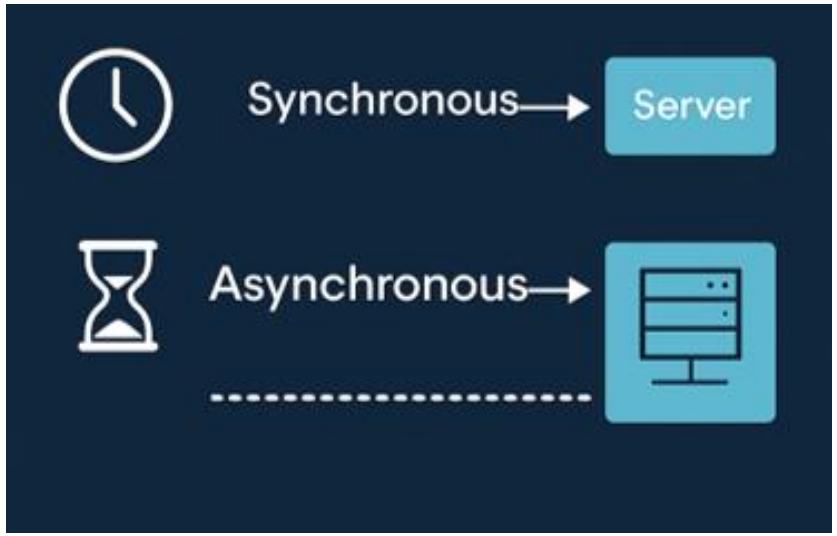


Live Demo

gRPC in a Polyglot Environment
Refer: Kingsly1921/labview-grpc-polyglot-demo: LabVIEW gRPC examples and a polyglot demo for building scalable multi-language systems.

Best Practices for Clean APIs

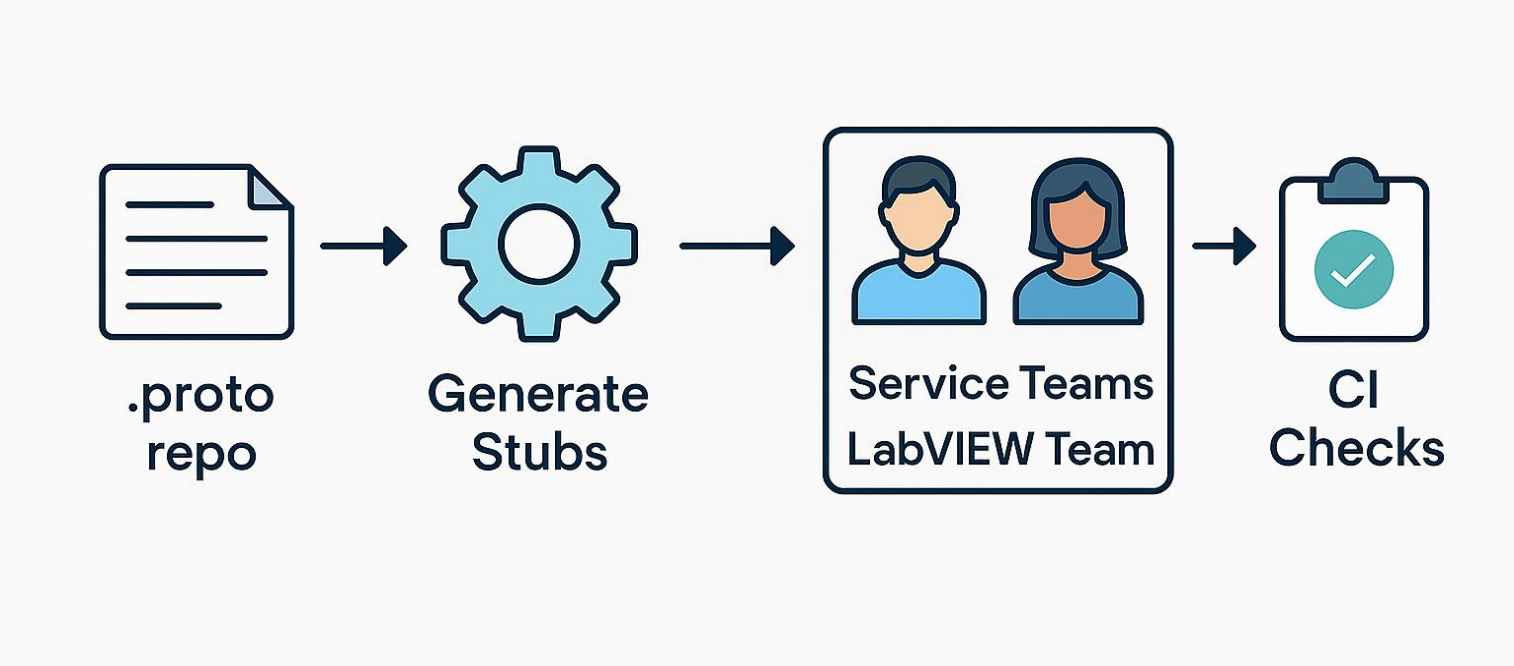
- Keep messages small and meaningful.
- Create message wrappers in LabVIEW for easy message updates.
- Asynchronous execution wherever necessary with limited calls:
  - Unary – to avoid server blocking any other calls.
  - Stream – to handle parallel streams from various clients.
- Keep logic in code, not in the .proto.
- Add error codes and clear status messages in responses.
- Document each RPC method and message.



- Implement a scaling service-based architecture and assign each service to separate async listeners.
- Properly close all the open services and RPC calls.

Team Workflow That Scales

- Contracts-first: agree on .proto before coding.
- Put .proto in a shared repo with reviews like code.
- Generate language stubs automatically for each team.
- CI validates compatibility whenever .proto changes.

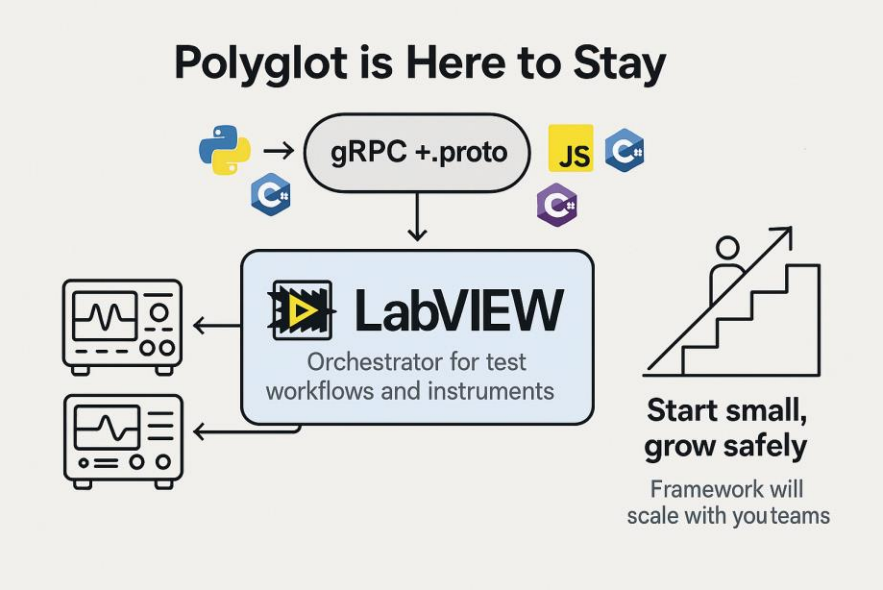


Where This Shines (Real Uses)

- Add AI/ML to a test sequence without touching LabVIEW logic.
- Offload heavy DSP/simulation to optimized native code.
- Integrate with enterprise/cloud systems through modern services.
- Let vendors deliver plugins in their preferred language.
- Scaling the communication with product addition.

Key Takeaways

- Polyglot is here to stay—use the best tool for each job.
- LabVIEW remains the orchestrator for test workflows and instruments.
- gRPC + .proto gives you speed, structure, and stability across languages.
- Start small, grow safely, and your framework will scale with your teams.



References

- gRPC (https://grpc.io/)
- ni/grpc-labview: gRPC client and server support for LabVIEW (https://github.com/ni/grpc-labview)
- Protocol Buffers Documentation (https://protobuf.dev/)
- protocolbuffers/protobuf: Protocol Buffers - Google's data interchange format (https://github.com/protocolbuffers/protobuf)

Thank you

G
DEV
CON
ASIA